Guía para Desarrolladores — Kumi TTPN Admin V2

> Documento de onboarding técnico. Lee esto antes de tocar código. Última actualización: 2026-04-17

Índice

1. Estructura del monorepo
2. BACKEND — Rails API
   - Flujo de autenticación
   - Conceptos transversales
   - Concerns de modelo
   - Concerns de controlador
   - Services
   - Helpers
   - Modelos importantes
   - Jobs Sidekiq
   - Rake tasks de datos
3. FRONTEND — Quasar/Vue 3
   - Patrón de página obligatorio
   - Composables genéricos
   - Composables de dominio
   - Services
   - Stores Pinia
   - Componentes globales reutilizables
   - Router y guards
   - Sistema de privilegios
   - Caché de catálogos
4. Reglas y anti-patrones

Estructura del monorepo

```
Kumi TTPN Admin V2/
├── ttpngas/           → Rails 7 API (Ruby)
├── ttpn-frontend/     → Quasar/Vue 3 PWA
├── ttpn_n8n/          → Automatizaciones n8n
├── ttpn_php/          → Código PHP legado (no tocar)
└── docker-compose.yml
```

BACKEND — Rails API

Flujo de autenticación

Hay **dos sistemas** de autenticación que coexisten:

Sistema	Usado por	Cómo funciona
Devise (HTML)	`ApplicationController`	Para rutas HTML legadas. No aplica al frontend SPA.
JWT manual	`Api::V1::BaseController`	Para toda la API. Ver flujo abajo.

Flujo JWT en `Api::V1::BaseController`:

1. `before_action :authenticate_with_jwt` lee el header `Authorization: Bearer <token>`.
2. Intenta decodificar como JWT (`HS256`, `secret_key_base`). Payload: `{ user_id, role, jti, exp }`.
3. Verifica que `decoded['jti'] == user.jti` (permite revocar tokens con `user.revoke_jwt!`).
4. Si falla el JWT, intenta como **API Key** (`ApiKey.authenticate(token)`).
5. `before_action :ensure_authenticated_user` bloquea con 401 si ninguno de los dos funcionó.
6. `current_user` devuelve el `User` o el `ApiUser` según el caso.

Métodos en `BaseController` que todo controlador hereda:

- `set_business_unit_id` → sadmin puede sobrescribir via `?business_unit_id=N`; usuario normal tiene BU fija.
- `current_ability` → si es API Key, ability de solo lectura (`can :read, :all`); si es usuario, delega a `Ability`.
- `require_admin!` → verifica `current_user.sadmin?`, responde 403 si no.
- `handle_error(message, status)` → respuesta de error estandarizada.

Conceptos transversales

Antes de agregar lógica nueva, entender estos mecanismos globales:

`Current` — `app/models/current.rb`

`ActiveSupport::CurrentAttributes`. Thread-safe, se resetea entre requests.

```
Current.user           # usuario autenticado del request
Current.business_unit  # BU del usuario (se asigna automáticamente al asignar .user=)
Current.role           # rol del usuario
Current.import_mode    # true durante imports masivos → omite callbacks pesados
Current.cuadre_in_progress  # true durante el cuadre booking↔TC → evita loops
```

Regla: nunca pases `current_user` o `business_unit` como parámetro entre métodos internos. Usa `Current`.

Business Unit como filtro universal

Casi todos los modelos tienen un scope `business_unit_filter` que usa `Current.business_unit`.

El interceptor de axios del FE agrega `?business_unit_id=N` automáticamente en todos los requests. El `BaseController` setea `Current.business_unit` con ese valor (o el del usuario si no es sadmin).

Regla: al crear un modelo nuevo que deba filtrarse por BU, agrega el scope `business_unit_filter` y llámalo en el `index` del controlador.

`sadmin` — qué es y qué NO es

`sadmin` es un flag booleano en la tabla `users`. No es un rol. Significa acceso de soporte/desarrollador:

- Puede cambiar de BU con `?business_unit_id=N`.
- En `build_privileges` obtiene acceso total a todos los módulos.
- Solo deben tenerlo los usuarios con rol `sistemas`.
- La migración `20260417000001_fix_sadmin_for_sistemas_role.rb` lo mantiene sincronizado.

Concerns de modelo

`app/models/concerns/` — no duplicar estas funcionalidades.

`Auditable`

Agrega tracking de `created_by` / `updated_by` automáticamente.

```
include Auditable
# Agrega: belongs_to :creator, belongs_to :updater
# Callbacks: before_create :set_creator, before_save :set_updater
# Scopes: created_by, updated_by, recent_changes
# Métodos: created_by_name, updated_by_name, audit_trail
```

Úsalo en cualquier modelo que tenga columnas `created_by_id` / `updated_by_id`. Depende de `Current.user`.

`Cacheable`

Cache automático con `Rails.cache` para modelos que cambian poco.

```
include Cacheable
self.cache_ttl = 1.hour  # optional, default 30 min

# Agrega: after_create/update/destroy :clear_cache
# Métodos de clase: cached_all, cached_active, cached_find(id),
cached_by_company(id), clear_all_cache
# Método de instancia: clear_cache
```

Úsalo en catálogos estáticos (tipos de vehículo, puestos, etc.). `Vehicle` lo usa con TTL de 1 hora.

`Cuadrable`

Comparte `match_fields_changed?` entre `TtpnBooking` y `TravelCount`.

```
include Cuadrable
MATCH_FIELDS = %w[fecha hora vehicle_id client_id ...]  # debe declararse en el
modelo
```

Solo aplica a los modelos del cuadre. No agregar a otros.

`SequenceSynchronizable`

Resuelve secuencias PK rotas por INSERTs directos desde apps Android/PHP.

```
include SequenceSynchronizable
# Agrega: before_create :sync_id_sequence
# Método de clase: sync_sequence! (para uso manual post-import)
```

Úsalo en modelos que reciben INSERTs directos desde sistemas externos (no desde la API Rails).

Concerns de controlador

`app/controllers/concerns/`

`ApiKeyAuthorizable`

Permite autenticación con API Key cuando no hay sesión JWT.

```
before_action :authenticate_with_api_key_or_user!
before_action :authorize_api_key_permissions!
# Helpers: using_api_key?, current_api_key
```

Ya incluido en `BaseController`. No incluir manualmente en controladores hijos.

`TtpnBookingsFilterable`

Centraliza filtros de fecha y utilidades de import para `TtpnBookingsController`.

```
include TtpnBookingsFilterable
# Métodos: apply_date_filters_listing(scope), apply_date_filters_stats(scope)
#          normalize_booking(booking), serialize_passenger(p)
#          open_spreadsheet(file)  → detecta CSV/XLS/XLSX y devuelve Roo
```

Solo aplica a `TtpnBookingsController` y su contexto.

Services

`app/services/` — encapsulan lógica de negocio compleja. Los controladores solo llaman al service, no implementan lógica.

`TtpnCuadreService` — `app/services/ttpn_cuadre_service.rb`

Cuadre bidireccional `TtpnBooking ↔ TravelCount`.

```
TtpnCuadreService.new(booking_or_travel).buscar_travel(booking)
TtpnCuadreService.new(booking_or_travel).buscar_booking(travel)
TtpnCuadreService.new(...).vincular(booking, travel)
TtpnCuadreService.new(...).desvincular_travel(travel_count_id)
TtpnCuadreService.new(...).desvincular_booking(booking_id)
```

Dos niveles de matching: Nivel 1 = `clv_servicio` exacta, Nivel 2 = función SQL con ventana ±15 min.

`TtpnDataService` — `app/services/ttpn_data_service.rb`

Query SQL compleja para el dashboard TTPN (viajes por sucursal de cliente).

```
TtpnDataService.new(params, current_user, branch_office_id).call { |progress| ... }
# Devuelve: { period: {...}, compare: {...} }
```

`DashboardDataService` — `app/services/dashboard_data_service.rb`

Query para el dashboard principal (viajes facturados por cliente, mes, tipo de vehículo).

```
DashboardDataService.new(params, current_user, business_unit_id).call { |progress|
... }
# Usa la función PostgreSQL cobro_fact() para calcular montos.
```

`Gasoline::EmployeeAssignment` — `app/services/gasoline/employee_assignment.rb`

Resuelve el `employee_id` vigente en un vehículo en una fecha/hora dada. Fallback: empleado CLV `'00000'`.

```
Gasoline::EmployeeAssignment.call(neconomico:, fecha:, hora: nil)
# → Integer (employee_id)
```

Regla: nunca uses el ID 63 hardcodeado como fallback. Usa siempre `Employee.find_by(clv: '00000')&.id`.

`FuelPerformance::VehicleCalculator` — `app/services/fuel_performance/vehicle_calculator.rb`

```
FuelPerformance::VehicleCalculator.call(fecha_inicio:, fecha_fin:, vehicle_id: nil)
# Con vehicle_id: métricas detalladas de un vehículo.
# Sin vehicle_id: array de todos los vehículos ordenados por rendimiento.
```

`FuelPerformance::PerformersRanker` — `app/services/fuel_performance/performers_ranker.rb`

```
FuelPerformance::PerformersRanker.call(fecha_inicio:, fecha_fin:)
# → { best: [...top 10], worst: [...bottom 10] }
```

`PayrollSvc::WeekCalculator` — `app/services/payroll_svc/week_calculator.rb`

Semana de nómina vigente (arranca el día configurado en `KumiSetting.dia_pago`, hora en `hora_corte`).

```
PayrollSvc::WeekCalculator.call(business_unit_id:)
# → { start_date:, start_time:, end_date:, end_time: }

PayrollSvc::WeekCalculator.apply_to_scope(scope, business_unit_id:)
# → aplica WHERE directamente a un scope AR
```

`PayrollSvc::ReportQuery` — `app/services/payroll_svc/report_query.rb`

```
PayrollSvc::ReportQuery.call(start_datetime:, end_datetime:)
# → Array de Employee con atributos: :puesto, :cost_viajes, :cont_viajes,
:deducciones
# Solo incluye choferes y coordinadores. Excluye empleado '00000'.
```

`Alerts::DispatcherService` — `app/services/alerts/dispatcher_service.rb`

Orquesta el envío de alertas: email, push FCM a usuarios internos, y ActionCable.

```
Alerts::DispatcherService.new(alert).call
```

Helpers

Los helpers en `app/helpers/` que tienen lógica real (los demás son stubs vacíos):

`TtpnBookingsHelper`

- `obtener_empleado(vehiculo, fecha_entera)` — query a función PostgreSQL `asignacion()`.
- `busca_en_travel(...)` — Nivel 2 del cuadre (ventana de tiempo).

Incluido en `TtpnBooking` y `TtpnCuadreService`. No duplicar esta lógica.

`TravelCountsHelper`

- `busca_en_booking(...)` — Nivel 2 del cuadre, dirección TravelCount → Booking.

Modelos importantes

`User` — `app/models/user.rb`

Método/Scope	Qué hace
`sadmin?`	Flag de soporte/desarrollador. Acceso total.
`role?(type)`	Compara `role.nombre` con el string dado.
`generate_jwt(exp)`	Genera token JWT firmado.
`revoke_jwt!`	Regenera el JTI, invalida todos los tokens existentes del usuario.
`build_privileges`	Si sadmin → acceso total. Si no → delega a `role.privileges_hash`.
`business_unit_filter` scope	sadmin ve todos; usuario normal filtra por su BU.

`Employee` — `app/models/employee.rb`

Método/Scope	Qué hace
`fecha_inicio_actual`	Último movimiento de ALTA o REINGRESO.
`years_worked(hasta_fecha)`	Años trabajados desde el último ALTA.
`dias_vacaciones_correspondientes(fecha)`	Delega a `KumiSetting`.
`dias_vacaciones_pendientes(fecha)`	Correspondientes - tomados.
`business_unit_filter` scope	**Siempre** filtra por BU. No hay escape para sadmin.

El empleado con `clv: '00000'` es el empleado "Sin Chofer" (placeholder). Nunca hardcodear su ID.

`Vehicle` — `app/models/vehicle.rb`

Incluye `Auditable` + `Cacheable` (TTL 1h). Usa FriendlyId con `:clv`.

`business_unit_filter` → sadmin ve todo; usuario normal filtra vía join: `vehicles → concessionaires → business_units`.

`TtpnBooking` — `app/models/ttpn_booking.rb`

El modelo más complejo. Sus callbacks hacen mucho trabajo automático:

Callback	Cuándo	Qué hace
`before_validation :extra_campos`	Siempre	Calcula `clv_servicio`, determina `employee_id`, limpia cuadre previo si cambiaron `MATCH_FIELDS`.
`before_create :statuses`	Al crear	Busca cuadre con TravelCount via `TtpnCuadreService`.
`after_create :create_actualiza_tc`	Al crear	Actualiza TravelCount con el `booking_id`.
`before_update :update_borra_tc`	Al actualizar	Si cambiaron `MATCH_FIELDS`, desvincula TC anterior y re-cuadra. Usa `Current.cuadre_in_progress`.
`after_destroy :borra_tc_destroy`	Al destruir	Desvincula TC.
`after_save :cuenta_pasajeros`	Al guardar	Actualiza `passenger_qty` con `update_all` (bypass callbacks).

Validación `sin_duplicado_en_15_minutos`: evita duplicados, no aplica a `creation_method: :automatico`.

`self.find_within_window(client_id:, fecha:, vehicle_id:, hora:, minutes: 15)` — busca bookings en ventana de tiempo.

Regla: cuando importes o crees bookings en masa, activa `Current.import_mode = true` para omitir callbacks pesados.

`ApiKey` — `app/models/api_key.rb`

```
ApiKey.authenticate(key)        # busca, valida, toca last_used_at
key.can?(resource, action)      # verifica permissions jsonb
key.revoke!
key.regenerate!
key.active_and_valid?
```

`AVAILABLE_PERMISSIONS` y `RESOURCE_NAMES_ES` son el catálogo completo de ~45 recursos con sus 4 acciones.

`Current` — `app/models/current.rb`

Ver sección Conceptos transversales. No crear variables de hilo por fuera de `Current`.

`Ability` — `app/models/ability.rb` (CanCanCan)

Roles: `:sistemas`, `:admin`, `:rh`, `:coordinador`, `:rh_tulpe`, `:coordinador_tulpe`, `:capturista`, `:as_direccion`, `:mantenimiento`, `:monitoreo`.

Al agregar un modelo nuevo, revisar `Ability` para agregar los permisos correspondientes a cada rol.

Jobs Sidekiq

Todos los jobs pesados son asíncronos. El frontend sigue el patrón: **initiate → job_id → poll status**.

Job	Trigger	Hace
`DashboardCalculationJob`	`DashboardController#initiate_load`	Ejecuta `DashboardDataService`, reporta progreso.
`TtpnCalculationJob`	Similar	Dashboard TTPN.
`DashboardExportJob`	`DashboardController#export`	Genera export del dashboard.
`TtpnBookingImportJob`	`TtpnBookingsController#import`	Import masivo desde Excel. Activa `Current.import_mode`.
`AlertDispatchJob`	Generado por alertas	Ejecuta `Alerts::DispatcherService`.
`DocExpirationCheckJob`	Cron diario	Revisa documentos por vencer, crea `Alert`, encola `AlertDispatchJob`. Tiene `already_alerted?` para evitar duplicados.
`DeactivateExpiredVersionsJob`	Cron	Desactiva versiones de app expiradas.

Rake tasks de datos

En `lib/tasks/`. Solo para uso en consola/deploy, no desde la app:

Task	Propósito
`reset_sequences`	Resetea secuencias PK de PostgreSQL (complemento de `SequenceSynchronizable`).
`backfill_clv_servicio`	Rellena `clv_servicio` en `TtpnBooking` histórico.
`backfill_travel_counts_clv_servicio`	Lo mismo para `TravelCount`.
`cleanup_ttpn_booking_passengers`	Limpieza de pasajeros huérfanos.
`setup_v2_modules`	Setup inicial de módulos y privilegios de la V2.

FRONTEND — Quasar/Vue 3

Patrón de página obligatorio

Toda página sigue: **Page orquestadora + composable(s) + componentes especializados**.

```
pages/
└── MiModulo/
    ├── MiModuloPage.vue       ← orquestador: solo template + instanciar composables
    ├── useMiModuloData.js     ← lógica de datos (fetch, CRUD, estado)
    ├── useMiModuloCatalogs.js ← carga de catálogos necesarios
    └── components/
```

```
          ├── MiFormDialog.vue
          └── MiDetallePanel.vue
```

Para páginas CRUD simples (catálogos):

- Usar `useCrud` + `AppTable` + `FilterPanel`.
- El composable Page no tiene lógica propia, solo conecta.

Para páginas complejas:

- Dividir en composables por dominio (`useData`, `useCatalogs`, `useExport`, etc.).
- El componente Page es solo el orquestador: instancia composables, pasa datos via props.

Composables genéricos

No reimplementar lo que ya existe.

`useCrud` — `src/composables/useCrud.js`

El composable más importante. Encapsula todo el ciclo CRUD estándar.

```
const {
  items, form, loading, saving, dialog,
  editingItem, isEditMode,
  fetchData, openDialog, closeDialog, save
} = useCrud({
  service:       miService,
  resourceName:  'mi_recurso',  // clave para el payload Rails { mi_recurso: form }
  formDefault:   { nombre: '', activo: true },
  createMsg:     'Registro creado',
  updateMsg:     'Registro actualizado'
})
```

`save` detecta automáticamente si es crear o editar vía `isEditMode`.

`useFilters` — `src/composables/useFilters.js`

```
const { filters, showFilters, activeFiltersCount, toggleFilters, clearFilters } =
  useFilters({ search: '', activo: null, fecha_inicio: '', fecha_fin: '' })

watch(filters, fetchData, { deep: true })
```

NO hace llamadas a API. Solo gestiona estado de filtros.

`useNotify` — `src/composables/useNotify.js`

```
const { notifyOk, notifyError, notifyApiError } = useNotify()

notifyApiError(err, 'Error al guardar')  // extrae automáticamente
error.response.data.errors
```

usePrivileges — src/composables/usePrivileges.js

```
const { canCreate, canEdit, canDelete, canImport, canExport } =
usePrivileges('mi_modulo')

// En template:
// v-if="canCreate()"
```

useSelectFilter — src/composables/useSelectFilter.js

Para el handler `@filter` de `q-select` con búsqueda en arrays pre-cargados.

```
const { filteredOptions, filterFn } = useSelectFilter(itemsRef, ['nombre', 'clv'])
// <q-select :options="filteredOptions" @filter="filterFn" />
```

useDateFormat — src/composables/useDateFormat.js

```
const { today, daysAgo, toDateStr, toTimeStr, toDateTime, currentTime } =
useDateFormat()
```

Usa hora local (no UTC). Siempre usar este composable para formatear fechas — no usar `new Date()` ni `dayjs` directamente.

useDropdownCache — src/composables/useDropdownCache.js

Para catálogos específicos de un módulo con TTL y soporte de ETag/304.

```
const { data, loading, load, invalidate } = useDropdownCache('mi_catalogo', fetchFn,
{ ttl: 5 * 60 * 1000 })
await load()
```

Para catálogos compartidos entre módulos, usar `useCatalogsStore` (ver Stores).

useBusinessUnitContext — src/composables/useBusinessUnitContext.js

```
const { isSuperAdmin, selectedBusinessUnit, loadBusinessUnits, selectBusinessUnit } =
  useBusinessUnitContext()
```

Persiste selección en `localStorage` con clave `selected_business_unit_id`.

usePayrollSettings — src/composables/usePayrollSettings.js

```
const { payrollSettings, loadSettings } = usePayrollSettings()
await loadSettings()
// payrollSettings.value.dia_pago, .hora_corte, .periodo
```

Composables de dominio

Composables específicos para módulos complejos. No son genéricos:

Composable	Módulo	Responsabilidad
`useTtpnBookingForm`	Captura bookings	Estado del formulario, validaciones, carga para edición, filtros de vehículos/choferes
`useTtpnBookingImport`	Import bookings	Lógica de importación masiva desde Excel
`useDashboardData`	Dashboard	Comunicación con jobs asíncronos, polling de estado
`useDashboardPeriod`	Dashboard	Gestión del período de análisis
`useTtpnData`	Dashboard TTPN	Similar a `useDashboardData`
`useTravelCountsData`	TravelCounts	Datos y CRUD de conteos de viaje

Composables orquestadores — coordinan carga de múltiples catálogos para una página:

Composable	Carga
`useVehiclesOrchestrator`	vehicleTypes, concessionaires, vehicleDocTypes
`useEmployeesOrchestrator`	vehicleTypes, concessionaires, laborTypes, businessUnits, driversLevels, employeeDocTypes, movementTypes
`useClientsOrchestrator`	catálogos para clientes

Composables de dropdowns — `src/composables/dropdowns/`:

Uno por catálogo: `useBusinessUnitsDropdown`, `useClientsDropdown`, `useConcessionairesDropdown`, `useDriversLevelsDropdown`, `useEmployeeDocTypesDropdown`, `useEmployeeMovementTypesDropdown`, `useLaborTypesDropdown`, `useTtpnServicesDropdown`, `useVehicleDocTypesDropdown`, `useVehicleTypesDropdown`.

Services (FE)

`src/services/` — wrappers de axios. Cada método retorna una promesa.

Archivo	Exporta
`bookings.service.js`	`bookingsService`, `travelCountsService`, `discrepanciesService`, `invoicingsService`
`employees.service.js`	`employeesService`, `deductionsService`, `incidencesService`, `vacationsService`, `employeeDocumentsService`, `aguinaldosService`
`vehicles.service.js`	`vehiclesService`, `vehicleChecksService`, `vehicleAsignationsService`, `vehicleTypePricesService`, `reviewPointsService`
`catalogs.service.js`	`servicesService`, `serviceTypesService`, `concessionairesService`, `suppliersService`, `driversLevelsService`, `vehicleTypesService`, `laborTypesService`, `businessUnitsService`, `employeeDocTypesService`, `employeeMovementTypesService`, `incidenceTypesService`, `vehicleDocumentTypesService`

Archivo	Exporta
`users.service.js`	`usersService`, `rolesService`, `privilegesService`, `apiUsersService`, `apiKeysService`, `versionsService`
`alerts.service.js`	`alertsService`, `alertRulesService`, `alertContactsService`
`dashboard.service.js`	`dashboardService` (initiateLoad, loadStatus, export, exportStatus)
`gas.service.js`	`gasChargesService`, `gasolineChargesService`, `gasStationsService`, `fuelPerformanceService`
`payroll.service.js`	`payrollsService`, `payrollReportsService`
`settings.service.js`	`payrollSettingsService`, `vacationSettingsService`

Regla: nunca hagas llamadas `api.get(...)` directamente en un componente o composable. Siempre a través del service correspondiente.

Stores Pinia

`src/stores/`

`auth-store.js`

```
const authStore = useAuthStore()
authStore.user          // objeto de usuario o null
authStore.isAuthenticated  // computed

authStore.login(email, password)  // POST login, guarda JWT, carga privilegios
authStore.logout()                // DELETE logout, limpia todo, invalida catálogos
```

Persiste `user` en localStorage bajo clave `'auth'`.

`privileges-store.js`

```
const privilegesStore = usePrivilegesStore()
privilegesStore.canAccess('modulo_key')
privilegesStore.canCreate('modulo_key')
privilegesStore.canEdit('modulo_key')
privilegesStore.canDelete('modulo_key')
privilegesStore.canImport('modulo_key')
privilegesStore.canExport('modulo_key')
```

Estructura de cada módulo: `{ can_access, can_create, can_edit, can_delete, can_clone, can_import, can_export }`.

Usar `usePrivileges('modulo_key')` (composable) en lugar de acceder al store directamente desde páginas.

`catalogs-store.js`

```
const catalogsStore = useCatalogsStore()
await catalogsStore.load('vehicleTypes')
```

```
  await catalogsStore.loadMany(['vehicleTypes', 'concessionaires'])
  catalogsStore.data.vehicleTypes   // array
  catalogsStore.invalidate('vehicleTypes')
  catalogsStore.invalidateAll()     // se llama en logout
```

Catálogos disponibles: `vehicleTypes`, `vehicleDocTypes`, `concessionaires`, `laborTypes`, `businessUnits`, `driversLevels`, `employeeDocTypes`, `movementTypes`, `ttpnServices`, `roles`.

`load(key)` es idempotente: no hace request si ya está cargado (usar `force: true` para forzar).

Componentes globales reutilizables

No reimplementar. Siempre buscar aquí primero.

`AppTable` — `src/components/AppTable.vue`

Tabla universal. Altura dinámica (llena el viewport). Paginación interna (default 20/página).

```
<AppTable :rows="items" :columns="columns" :loading="loading">
  <template #cell-status="{ row }">
    <q-badge :color="row.active ? 'positive' : 'grey'">{{ row.active ? 'Activo' :
'Inactivo' }}</q-badge>
  </template>
  <template #cell-actions="{ row }">
    <q-btn flat icon="edit" @click="openDialog(row)" />
    <q-btn flat icon="delete" @click="handleDelete(row)" />
  </template>
</AppTable>
```

Slot por columna: `#cell-{column.name}`. Resetea a página 1 automáticamente cuando cambia `rows`.

`FilterPanel` — `src/components/FilterPanel.vue`

Panel colapsable. Al expandirse/colapsarse emite `window.resize` para que `AppTable` recalcule.

```
<FilterPanel v-model="showFilters" :active-count="activeFiltersCount"
@clear="clearFilters">
  <div class="col-12 col-md-3">
    <q-input v-model="filters.search" label="Buscar" outlined dense />
  </div>
  <div class="col-12 col-md-3">
    <q-select v-model="filters.tipo" :options="tipos" label="Tipo" outlined dense
clearable />
  </div>
</FilterPanel>
```

`PageHeader` — `src/components/PageHeader.vue`

```
<PageHeader title="Vehículos" subtitle="Gestión de unidades">
  <template #actions>
    <q-btn label="Agregar" icon="add" color="primary" @click="openDialog()" />
```

```
    </template>
  </PageHeader>
```

`CatalogManager` — `src/components/CatalogManager.vue`

Para páginas de catálogos simples que solo necesitan lista + form:

```
<CatalogManager
  title="Tipos de Vehículo"
  :columns="columns"
  :rows="items"
  :loading="loading"
  :boolean-columns="['activo']"
  ...
/>
```

`AlertBell` — `src/components/AlertBell.vue`

Campanita en el header. Badge con conteo de no leídas. No instanciar fuera de `MainLayout`.

`AppProgressDialog` — `src/components/AppProgressDialog.vue`

Para operaciones asíncronas con polling (dashboard, exports, imports). Muestra progreso porcentual.

Router y guards

`src/router/routes.js` — todas las rutas autenticadas son hijos de `/` con `meta: { requiresAuth: true }`.

`src/router/index.js` — guard `beforeEach`:

```
requiresAuth && !isAuthenticated  →  /login
en /login && isAuthenticated      →  /
requiresSadmin && !user.sadmin    →  /
```

`requiresSadmin: true` actualmente solo aplica a `/api-access`.

Regla: para agregar una ruta nueva solo accesible a sadmin, agregar `meta: { requiresSadmin: true }`. Para rutas protegidas por privilegio, controlar el acceso dentro de la página con `usePrivileges`, no en el router.

Sistema de privilegios

1. En login, la API devuelve el hash `privileges` en la respuesta.
2. `authStore.login` lo pasa a `privilegesStore.setPrivileges(privileges)`.
3. El store persiste en localStorage (Pinia persist plugin).
4. En páginas: `usePrivileges('module_key')` — retorna funciones que consultan el store.
5. `module_key` en snake_case: `'clients_directory'`, `'ttpn_bookings'`, `'fuel_performance'`, etc.
6. En template: `v-if="canCreate()"` para ocultar botones según permisos.
7. Sadmin siempre obtiene acceso total (el backend lo construye en `build_privileges`).

Caché de catálogos

Dos niveles:

Mecanismo	Cuándo usarlo	Alcance
`useCatalogsStore` (Pinia)	Catálogos compartidos entre varios módulos	Global, persiste entre navegaciones, se invalida en logout
`useDropdownCache`	Catálogos específicos de un módulo	Module singleton, TTL configurable, soporte ETag

Regla: si un catálogo se usa en más de un módulo, va al store. Si es específico de un módulo, usa `useDropdownCache`.

Reglas y anti-patrones

Backend

❌ No hacer	✅ Hacer en su lugar
Hardcodear el ID del empleado `'00000'` (ej: `	
Poner lógica de negocio en controladores	Crear un Service en `app/services/`
Implementar auditoría (created_by/updated_by) manualmente	Incluir el concern `Auditable`
Usar `current_user` como parámetro entre métodos internos	Usar `Current.user`
Crear imports masivos sin desactivar callbacks	Activar `Current.import_mode = true`
Hacer INSERTs directos en tablas con relaciones sin verificar secuencias	Incluir `SequenceSynchronizable`
Duplicar la lógica de ventana de tiempo del cuadre	Usar `TtpnCuadreService`
Filtrar por BU manualmente en cada query	Usar el scope `business_unit_filter`

Frontend

❌ No hacer	✅ Hacer en su lugar
Llamar `api.get(...)` directamente en componentes	Usar el service correspondiente en `src/services/`
Re-implementar CRUD en cada página	Usar `useCrud`
Re-implementar manejo de filtros	Usar `useFilters`
Mostrar notificaciones con `$q.notify(...)` directamente	Usar `useNotify`
Cargar catálogos con fetch local en cada componente	Usar `useCatalogsStore.load(key)` o `useDropdownCache`
Formatear fechas con `new Date()` o `dayjs` directamente	Usar `useDateFormat`
Implementar tabla propia	Usar `AppTable`
Implementar panel de filtros propio	Usar `FilterPanel`

❌ No hacer	✅ Hacer en su lugar
Poner lógica de negocio en el componente Page	Extraerla a un composable
Mutar props directamente en Vue 3	Usar `:model-value` + `emit('update:modelValue', ...)`
Agregar rutas sadmin-only al guard del router	Agregar `meta: { requiresSadmin: true }` en routes.js